

Robot Framework

Open-source keyword-driven framework extensible through libraries for Web, API, desktop and other domains.

What it is and when to use it

Robot Framework uses tabular syntax and keywords, supporting readable and reusable high-level automation.

Official site and documentation

Use the official Robot Framework source as the primary reference: <https://robotframework.org/>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

Recommended virtual environment

Create a project folder, run `python -m venv .venv` and activate it. Use `.venv\Scripts\Activate.ps1` on PowerShell or `source .venv/bin/activate` on macOS/Linux.

First suite

Create a `.robot` file with Settings, Test Cases and Keywords and run `robot path/to/suite.robot`. Review `output.xml`, `log.html` and `report.html`.

Keywords and organization

Design business-oriented keywords and keep resources, variables and libraries predictable.

Libraries

Choose libraries according to the tested technology and project maturity.

CI/CD and reports

Install requirements, run robot and publish log/report/evidence artifacts.

Troubleshooting

Check PATH, active virtual environment, Python version and whether packages were installed into the correct interpreter.

How to start safely

Start from the official Robot Framework site and documentation. Confirm prerequisites, supported versions, operating system and required permissions. Learn in a controlled environment before using it in a critical workflow.

First QA-oriented use

Pick one small, verifiable task in Robot Framework, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where Robot Framework fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether Robot Framework still delivers value as the product evolves.

Common issues and troubleshooting

When Robot Framework fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.

Installation — prerequisites

Robot Framework is commonly installed with Python. Validate `python --version` and `python -m pip --version` and prefer a virtual environment.

Install Robot Framework

With the virtual environment active run `python -m pip install robotframework` and validate `robot --version`.

First test

Create `example.robot` with `*** Test Cases ***` and a simple Log keyword. Run `robot example.robot` and open generated `log.html/report.html`.

Libraries

Install only libraries required by the project and record versions in `requirements.txt` or equivalent.

Troubleshooting

If robot is not recognized, confirm the virtual environment is active and Scripts is available. You can also use `python -m robot --version`.